

PLP - Segundo Parcial - 1^{er} cuatrimestre de 2023

Este examen se aprueba obteniendo al menos dos ejercicios bien menos (B-) y uno regular (R). Las notas para cada ejercicio son: -, I, R, B-, B. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación Lógica

- a) Sea un alfabeto un conjunto de átomos cualesquiera, y sea una palabra en cierto alfabeto A una lista finita de átomos que pertenecen a A. Implementar el predicado palabra(+A, +N, -P), que genera las palabras en A cuya longitud es de exactamente N elementos. Tanto en este inciso como en el siguiente, no es obligatorio respetar el mismo orden de los ejemplos.

```
?- palabra([a,b,c], 4, P).  
P = [a,a,a,a] ;  
P = [a,a,a,b] ;  
...  
P = [b,c,a,b] ;  
...  
P = [c,c,c,c] ;  
false
```

- b) Sea una frase en cierto alfabeto A una lista finita de palabras no vacías en A. Implementar el predicado frase(+A, -F), que genera todas las posibles frases en A.

```
?- frase([q,u,e,s,o], F).  
F = [] ;  
...  
F = [[q,u,e],[e,s],[e,s,o]] ;  
...  
F = [[q,u,e,s,o,s]] ;  
...
```

- c) Indicar si el parámetro P del predicado palabra y el parámetro F del predicado frase son reversibles. Justificar la respuesta.

Ejercicio 2 - Resolución

- a) Representar en forma clausal las siguientes fórmulas de lógica de primer orden referidas a números naturales:
- $\forall X \forall Y (X = Y) \supset divide(X, Y) \wedge divide(Y, X)$ - Si dos números son iguales, se dividen mutuamente.
 - $\forall X \forall Y \forall Z (divide(X, Y) \wedge divide(Y, Z)) \supset divide(X, Z)$ - La relación de divisibilidad es transitiva.
- b) Usando el método de resolución demostrar la siguiente fórmula:
 $\forall X \forall Y (X = Y) \supset divide(X, X)$
- c) La resolución realizada en el punto anterior, ¿fue SLD? Justificar. Si no lo fue, ¿sería posible encontrar una resolución SLD para este conjunto de cláusulas? (No es necesario escribirla, solamente indicar por qué se puede saber que es posible o que no lo es.)

Ejercicio 3 - Objetos

a) Definir en Cálculo ς la clase *Pokemon*, cuyas instancias puedan responder a los mensajes *salud*, *atacar* y *evolucionar*. La salud inicial de los pokemons es 100, al atacar reducen en 1 la salud de su oponente y al evolucionar, su ataque se incrementa en 1, es decir, cuando un pokemon evoluciona, hace 1 más de daño a su oponente que antes de evolucionar. Tener en cuenta que los pokemons pueden evolucionar más de una vez.

b) Se definen los siguientes objetos:

```
def [ cantidadDePokemons  $\doteq$  cero,  
    adoptar =  $\varsigma(z)\lambda(x)z.cantidadDePokemons := z.cantidadDePokemons.succ$  ]  
pikachu  $\doteq$  Pokemon.new
```

cero es el objeto definido en clase, y se tiene que $cero.succ \longrightarrow uno'$. Mostrar cómo reduce $ash.adoptar(pikachu)$. Está permitido usar la regla APP vista en clase.

PLP - Recuperatorio del Segundo Parcial - 1^{er} cuatrimestre de 2023

Este examen se aprueba obteniendo al menos dos ejercicios bien menos (B-) y uno regular (R). Las notas para cada ejercicio son: -, I, R, B-, B. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras.

El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

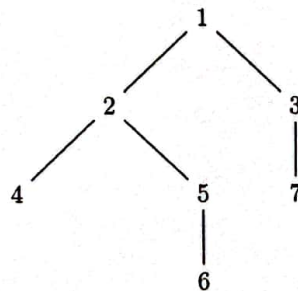
Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. No usar cut (!) ni predicados de alto orden como `setof`, con la única excepción de `not`. No está permitido utilizar estructuras auxiliares.

Trabajaremos con árboles binarios, usando `nil` y `bin(I, V, D)` para representarlos en Prolog. Como ejemplo, usaremos el siguiente hecho:

```
arbol(bin(bin(bin(nil,4,nil),2,bin(nil,5,bin(nil,6,nil))),1,bin(bin(nil,7,nil),3,nil))).
```

que representa el árbol:



- a) Definir el predicado `camino(+A,-C)` que es verdadero cuando `C` es un camino desde la raíz del árbol `A` hasta alguna de sus hojas. Por ejemplo:

```
?- arbol(A), camino(A,C).
C = [1,2,4];
C = [1,2,5,6];
C = [1,3,7];
false.
```

- b) Definir el predicado `caminoMasLargo(+A,-C)` que es verdadero cuando `C` es el camino más largo de `A`. Por ejemplo:

```
?- arbol(A), caminoMasLargo(A,C).
C = [1,2,5,6];
false.
```

- c) Definir el predicado `caminoUnicoDeLong(+A,+N,-C)` que es verdadero cuando `C` es el único camino de `A` que tiene longitud `N`. Por ejemplo:

```
?- arbol(A), caminoUnicoDeLong(A,4,C).
C = [1,2,5,6];
false.
?- arbol(A), caminoUnicoDeLong(A,3,C).
false.
```

Ejercicio 2 - Objetos

a) Sea el siguiente código escrito en JavaScript:

```
function Slime(energía) { this.energía = energía; }

Slime.prototype.actualizarEnergía = function(valor){ this.energía += valor; }

function SlimeSanador(energía, poder) {
  Slime.call(this, energía);
  this.poder = poder;
  this.curar = function(paciente) {
    paciente.actualizarEnergía(this.poder);
  }
}

let a = new SlimeSanador(10, 40);

Object.setPrototypeOf(SlimeSanador.prototype, Slime.prototype);

Slime.prototype.reproducirse = function() {
  if (this.energía >= 10) {
    let cría = Object.create(this);
    cría.energía = 10;
    this.actualizarEnergía(-10);
    return cría;
  }
}

let b = new Slime(10);
a.curar(b);
let c = b.reproducirse();
b.comidaFavorita = "gelatina"
c.películaFavorita = "los cazafantasmas"
```

Indicar qué mensajes (*energía*, *actualizarEnergía*, *poder*, *curar*, *reproducirse*, *comidaFavorita* o *películaFavorita*) pueden responder los objetos a, b y c al finalizar la ejecución de la última línea, en qué objeto/s está definido cada mensaje; y describir las cadenas de prototipos.

b) Sean las siguientes definiciones en cálculo Sigma:

$$\text{error} \stackrel{\text{def}}{=} []$$
$$\begin{aligned} \text{Lista} \stackrel{\text{def}}{=} [& \text{new} = \zeta(z) [\text{isEmpty} = \zeta(s) z.\text{isEmpty}(s), \\ & \text{cons} = \zeta(s) z.\text{cons}(s), \\ & \text{head} = \zeta(s) z.\text{head}(s), \\ & \text{tail} = \zeta(s) z.\text{tail}(s)], \\ & \text{isEmpty} = \lambda(s) \text{True}, \\ & \text{cons} = \lambda(s) \lambda(x) ((s.\text{isEmpty} := \text{False}).\text{head} := x).\text{tail} := s, \\ & \text{head} = \lambda(s) \text{error}, \\ & \text{tail} = \lambda(s) s] \end{aligned}$$

Considerar definidos los objetos *true*, *false* y *cero* vistos en clase. Definir la función *longitud* que calcule la longitud de una lista. Por ejemplo, tiene que valer

$$\text{longitud}(\text{Lista}.\text{new}.\text{cons}(\text{cero}).\text{cons}(\text{cero})) \longrightarrow \text{dos'}$$

Ejercicio 3 - Resolución

Considerar las siguientes definiciones en Prolog:

```
frutal(frutilla).  
frutal(banana).  
cremoso(banana).  
cremoso(americana).  
cremoso(frutilla).  
leGusta(X) :- frutal(X), cremoso(X).  
cucurucho(X,Y) :- leGusta(X), leGusta(Y).
```

y la consulta:

?- cucurucho(X,Y).

- a) Convertir la base de conocimientos y la consulta a forma clausal.
- b) Utilizar el método de resolución para obtener el resultado de la consulta.
- c) La resolución utilizada en el inciso anterior, ¿fue SLD? Justificar. En caso de ser SLD, ¿respeto el orden en que Prolog hubiera resuelto la consulta? ¿Por qué?

PLP - Segundo Parcial - 1^{er} cuatrimestre de 2024

#Orden	Nro. Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final

Este examen se aprueba obteniendo al menos dos ejercicios bien menos (B-) y uno regular (R). Las notas para cada ejercicio son: -, I, R, B-, B. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Resolución

a) Representar en forma clausal las siguientes fórmulas de lógica de primer orden:

- $\forall X.(\text{Radio}(X) \implies \text{Ruidoso}(X))$
Todas las radios son ruidosas.
- $\forall X.\forall Y.((\text{Posee}(X, Y) \wedge \text{Libro}(Y)) \implies \neg \exists Z.(\text{Posee}(X, Z) \wedge \text{Revista}(Z)))$
Toda persona que posee un libro no posee revistas.
- $\forall X.(\text{EsDormilon}(X) \implies \neg \exists Y.(\text{Posee}(X, Y) \wedge \text{Ruidoso}(Y)))$
Las personas que son dormilonas no poseen nada ruidoso.
- $\exists X.(\text{Posee}(\text{pepe}, X) \wedge (\text{Libro}(X) \vee \text{Radio}(X)))$
Pepe posee un libro o una radio.

b) Utilizando resolución, determinar si la siguiente fórmula es consecuencia del conjunto anterior:

$\text{EsDormilon}(\text{pepe}) \implies \neg \exists Z.(\text{Posee}(\text{pepe}, Z) \wedge \text{Revista}(Z))$
Si Pepe es dormilón, entonces no posee revistas.

Indicar la sustitución utilizada en cada paso. Es importante tener un plan, ya sea escrito o en la cabeza.

c) Mirando la resolución utilizada en el punto anterior, ¿fue SLD? Justificar.

Ejercicio 2 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. No usar cut (!) ni predicados de alto orden como **setof**, con la única excepción de **not**.

En este ejercicio ayudaremos a un grupo de jurados a analizar las notas de las materias que cursaron algunos estudiantes para presentarse a concurso. Se cuenta con el predicado **estudiante(?E)** que es verdadero cuando E es un estudiante, y con el predicado **notas(-XS)** que instancia en XS una lista de triplas correspondientes a las notas de los estudiantes en cada materia de la siguiente forma: **(Estudiante, Materia, Nota)**. En la lista de notas pueden aparecer aplazos, pero se asume que está bien formada (es decir, puede haber muchos aplazos para un estudiante en una materia, pero a lo sumo un aprobado). Se pide:

a) Definir el predicado **tieneMateriaAprobada(+E,+M)** que es verdadero cuando el estudiante E tiene la materia M aprobada (es decir, la nota es mayor o igual a 4).

b) Definir el predicado **eliminarAplazos(+NS,-L)** que es verdadero cuando NS es una lista de notas y L es la misma lista, pero eliminando los aplazos. Los aplazos sólo pueden eliminarse si el estudiante finalmente aprobó la materia. Por ejemplo:

```
?- eliminarAplazos([(juan,plp,3),(juan,plp,9),(maria,tlen,2)],L).
L = [(juan,plp,9),(maria,tlen,2)].
```

c) Definir el predicado **promedio(+A,-P)** que es verdadero cuando A es un estudiante, y P es el promedio de todas sus notas luego de eliminar los aplazos.

Sugerencia: armar una lista de notas de A.

d) Definir el predicado **mejorEstudiante(-A)** que es verdadero cuando A es el estudiante cuyo promedio es el más alto. Puede haber más de una solución en caso de que haya más de un estudiante con el promedio más alto. En este inciso no está permitido utilizar estructuras auxiliares.

Ejercicio 3 - Objetos y Deducción Natural

a) I. Considerar las siguientes definiciones:

```
Object subclass: A [  
  a: x b: y  
    ^ x a: (y c) b: self.  
  
  c  
    ^ 2.  
]  
  
A subclass: B [  
  a: x b: y  
    ^ y c + x value.  
  
  c  
    ^ 1.  
]  
  
B subclass: C [  
  a: x b: y  
    ^ x.  
  
  c  
    ^ [self a: super c b: self].  
]
```

Hacer una tabla donde se indique, en orden, cada mensaje se envía, qué objeto lo recibe, con qué colaboradores, en qué clase está el método respectivo, y cuál es el resultado final de cada colaboración tras ejecutar el siguiente código:

```
(A new) a: (B new) b: (C new)
```

II. Implementar un método para el mensaje `#divisores`, cuyo objeto receptor es un número entero, que devuelve una colección con sus divisores.

Sugerencia: utilizar el mensaje binario `#'\'` que devuelve el resto de la división entera entre dos números.

b) Demostrar en deducción natural que vale la siguiente fórmula sin usar principios de razonamiento clásicos:

$$\forall X.\forall Y.(\exists Z.(P(X,Z) \wedge P(Z,Y)) \implies \exists W.P(X,W))$$

PLP - Segundo Recuperatorio - 1^{do} cuatrimestre de 2024

#Orden	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
117.	64/22	Kerbs Octavio	B-	B-	B-	A

Este examen se aprueba obteniendo al menos dos ejercicios bien menos (B-) y uno regular (R). Las notas para cada ejercicio son: -, I, R, B-, B. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Resolución

a) Representar en forma clausal las siguientes fórmulas de lógica de primer orden:

$$1. \exists C. \forall X. (X \in a \Rightarrow X \leq C)$$

Existe una cota superior para el conjunto a.

$$2. \forall X. \forall Y. (X \leq Y \Rightarrow n(Y) \leq n(X))$$

Si $X \leq Y$, entonces $-Y \leq -X$.

$$3. \forall X. (X \in a \Leftrightarrow n(X) \in b)$$

X pertenece al conjunto a si y solo si -X pertenece al conjunto b.

b) Utilizando resolución, determinar si existe una cota inferior para el conjunto b:

$$\exists C. \forall X. (n(X) \in b \Rightarrow C \leq n(X)).$$

c) La resolución utilizada en el punto anterior, ¿fue SLD? Justificar.

Ejercicio 2 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. No usar cut (!) ni predicados de alto orden como setof, con la única excepción de not.

a) Implementar el predicado `generarCapicúas(-L)`, que genera todas las listas capicúas de números naturales (sin incluir al cero). Por ejemplo:

?- `generarCapicuas(L).`

`L = [1] ;`

`L = [2] ;`

`...`

`L = [1,3,1] ;`

`...`

`L = [2,3,3,2] ;`

`...`

b) El predicado anterior ¿es reversible? Justificar.

- c) Definir el predicado `tokenizar(+D,+F,-T)`, que es verdadero cuando D representa un diccionario de palabras conocidas (lista de listas de letras), F es una lista de letras sin espacios, y T es la lista de palabras en la que se puede partir la frase. El predicado debe generar todas las posibles maneras de dividir la frase en palabras conocidas. Por ejemplo:

```
?- tokenizar([[a],[a,b],[b,a],[c]], [a,b,a,c,a,b,a], T).
T = [[a],[b,a],[c],[a],[b,a]];
T = [[a],[b,a],[c],[a,b],[a]];
T = [[a,b],[a],[c],[a],[b,a]];
T = [[a,b],[a],[c],[a,b],[a]];
false.
```

- d) Definir el predicado `mayorCantPalabras(+D,+F,-T)` que es verdadero cuando T es una tokenización de la frase F con el diccionario D que tiene la mayor cantidad de palabras. En este inciso no está permitido utilizar estructuras auxiliares.

Ejercicio 3 - Objetos y Deducción Natural

- a) Considere las siguientes clases:

Object subclass: #A

```
m1
^[self eval].
```

```
eval
^self value.
```

```
value
^0.
```

A subclass: #B

```
m2
^[super eval].
```

```
m3
^self.
```

```
value
^1.
```

```
eval
^2.
```

Para cada una de las siguientes expresiones, hacer una tabla donde se indique, en orden, cada mensaje que se envía, qué objeto lo recibe, en qué clase está el método respectivo, y cuál es el resultado final de cada colaboración:

I) B new m1 value

II) B new m3 m2 value

- b) Implementar un *closure* que tome como parámetro una colección y devuelva los elementos de la colección que saben responder al mensaje `#ptff`.

Sugerencia: pueden utilizar el mensaje `#respondsTo:`, que toma un mensaje y devuelve si el objeto sabe responderlo.

- c) Demostrar en deducción natural que vale la siguiente fórmula, sin utilizar principios de razonamiento clásicos:

$$\exists X.P(X) \vee \forall Y.Q(Y) \Rightarrow \exists X.(Q(X) \vee P(X))$$

PLP - Segundo Parcial - 2^{do} cuatrimestre de 2024

#Orden	Nro. Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final

Este examen se aprueba obteniendo al menos dos ejercicios bien (B) y uno regular (R), y se promociona con al menos dos ejercicios muy bien (MB) y uno bien (B). Es posible obtener una aprobación condicional con un ejercicio muy bien (MB), uno bien (B) y uno insuficiente (I), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: -, I, R, B, MB, E. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar cut (!) ni predicados de alto orden como `setof`, con la única excepción de `not`.

- a) Definir el predicado `subsecuenciaCreciente(+L,-S)` que es verdadero cuando `S` es una subsecuencia estrictamente creciente de elementos de `L`. Notar que la secuencia respeta el orden de aparición en `L`. Por ejemplo:

```
?- subsecuenciaCreciente([4,8,1,9],S).
S = [4, 8, 9] ;
S = [4, 8] ;
S = [4, 9] ;
S = [4] ;
S = [8, 9] ;
S = [8] ;
S = [1, 9] ;
S = [1] ;
S = [9] ;
S = [] .
```

- b) Definir el predicado `subsecuenciaCrecienteMasLarga(+L,-S)` que es verdadero cuando `S` es la subsecuencia estrictamente creciente de mayor longitud de `L`. Puede haber más de un resultado. Por ejemplo:

```
?- subsecuenciaCrecienteMasLarga([5,6,7,2,8,1,2,3,4,5,7],S).
S = [1,2,3,4,5,7] ;
false.

?- subsecuenciaCrecienteMasLarga([5,6,7,2,8,0,2,3,4],S).
S = [5,6,7,8] ;
S = [0,2,3,4] ;
false.
```

- c) Definir el predicado `fibonacci(-X)` que instancia en `X` los números pertenecientes a la secuencia de Fibonacci. Por ejemplo:

```
?- fibonacci(X).
X = 1;
X = 1;
X = 2;
X = 3;
X = 5;
...
```

- d) ¿Es reversible el predicado anterior? Justificar.

Ejercicio 2 - Resolución

- a) Representar en forma clausal las siguientes fórmulas de lógica de primer orden, que tratan acerca de términos cerrados del cálculo lambda:

- $\forall T_1. \forall T_2. \forall M. \forall N. ((\text{Tipo}(M, T_1 \rightarrow T_2) \wedge \text{Tipo}(N, T_1)) \implies \text{Tipo}(\text{app}(M, N), T_2))$
Si el tipo de un término es $T_1 \rightarrow T_2$, y el tipo de otro término es T_1 , entonces el tipo de la aplicación es T_2 .
- $\exists M. \text{Tipo}(M, \alpha \rightarrow (\beta \rightarrow \gamma))$
Existe un término de tipo $\alpha \rightarrow (\beta \rightarrow \gamma)$.
- $\exists M. \text{Tipo}(M, \alpha \rightarrow \beta)$
Existe un término de tipo $\alpha \rightarrow \beta$.

- iv) $\exists M. \text{Tipo}(M, \alpha)$
Existe un término de tipo α .

Ayuda: se puede pensar en el constructor de tipos $\rightarrow$ como una función binaria, por ejemplo el término $T_1 \rightarrow T_2$ se puede pensar como $\text{flecha}(T_1, T_2)$. Además, α , β y γ son constantes distintas entre sí.

- b) Utilizando resolución, determinar si la siguiente fórmula es consecuencia del conjunto anterior:
 $\exists M. \text{Tipo}(M, \gamma)$ (*Existe un término de tipo γ*).
 Indicar la sustitución utilizada en cada paso. Es importante tener un plan (escrito o en la cabeza).
- c) ¿Fue SLD la resolución utilizada en el punto anterior? Justificar.

Ejercicio 3 - Inferencia y Deducción Natural

- a) Consideremos el Cálculo Lambda tipado extendido con árboles ternarios:

$$\tau ::= \dots \mid \text{AT}(\tau)$$

$$M ::= \dots \mid \text{TNil}_\tau \mid \text{Tern}(M, M, M, M) \mid \text{foldAT } M \triangleright \text{TNil} \rightsquigarrow M; \text{Tern}(x, ri, rm, rd) \rightsquigarrow M$$

Se extiende también el algoritmo de inferencia para árboles de la siguiente manera:

$$\mathbb{W}(\text{TNil}) \stackrel{\text{def}}{=} \emptyset \vdash \text{TNil}_X : \text{AT}(X) \quad \text{con } X \text{ variable fresca}$$

$$\mathbb{W}(\text{Tern}(U_1, U_2, U_3, U_4)) \stackrel{\text{def}}{=} ST_1 \cup ST_2 \cup ST_3 \cup ST_4 \vdash S(\text{Tern}(M_1, M_2, M_3, M_4)) : S\sigma_4$$

donde:

- $\mathbb{W}(U_1) = \Gamma_1 \vdash M_1 : \sigma_1$
- $\mathbb{W}(U_2) = \Gamma_2 \vdash M_2 : \sigma_2$
- $\mathbb{W}(U_3) = \Gamma_3 \vdash M_3 : \sigma_3$
- $\mathbb{W}(U_4) = \Gamma_4 \vdash M_4 : \sigma_4$
- $S = \text{mgu}(\{\sigma_2 \stackrel{?}{=} \text{AT}(\sigma_1), \sigma_3 \stackrel{?}{=} \sigma_2, \sigma_4 \stackrel{?}{=} \sigma_2\} \cup \{\tau \stackrel{?}{=} \rho \mid x : \tau \in \Gamma_i \wedge x : \rho \in \Gamma_j \wedge i, j \in \{1, 2, 3, 4\}\})$

$$\mathbb{W}(\text{foldAT } U_1 \triangleright \text{TNil} \rightsquigarrow U_2; \text{Tern}(x, ri, rm, rd) \rightsquigarrow U_3) \stackrel{\text{def}}{=} ST_1 \cup ST_2 \cup ST_{3'} \vdash S(\text{foldAT } M_1 \triangleright \text{TNil} \rightsquigarrow M_2; \text{Tern}(x, ri, rm, rd) \rightsquigarrow M_3) : S\sigma_2$$

donde:

- $\mathbb{W}(U_1) = \Gamma_1 \vdash M_1 : \sigma_1$
- $\mathbb{W}(U_2) = \Gamma_2 \vdash M_2 : \sigma_2$
- $\mathbb{W}(U_3) = \Gamma_3 \vdash M_3 : \sigma_3$
- $\tau_x = \begin{cases} \alpha_1 \text{ si } x : \alpha_1 \in \Gamma_3, \\ \text{variable fresca si no} \end{cases}$
- $\tau_{ri} = \begin{cases} \alpha_2 \text{ si } ri : \alpha_2 \in \Gamma_3, \\ \text{variable fresca si no} \end{cases}$
- $\tau_{rm} = \begin{cases} \alpha_3 \text{ si } rm : \alpha_3 \in \Gamma_3, \\ \text{variable fresca si no} \end{cases}$
- $\tau_{rd} = \begin{cases} \alpha_4 \text{ si } rd : \alpha_4 \in \Gamma_3, \\ \text{variable fresca si no} \end{cases}$
- $\Gamma_{3'} = \Gamma_3 \ominus \{x, ri, rm, rd\}$
- $S = \text{mgu}(\{\sigma_1 \stackrel{?}{=} \text{AT}(\tau_x), \sigma_2 \stackrel{?}{=} \sigma_3, \tau_{ri} \stackrel{?}{=} \sigma_2, \tau_{rm} \stackrel{?}{=} \sigma_2, \tau_{rd} \stackrel{?}{=} \sigma_2\} \cup \{\tau \stackrel{?}{=} \rho \mid x : \tau \in \Gamma_i \wedge x : \rho \in \Gamma_j \wedge i, j \in \{1, 2, 3'\}\})$

Además, se considera extendido el algoritmo para la suma de naturales:

$$\mathbb{W}(U_1 + U_2) \stackrel{\text{def}}{=} ST_1 \cup ST_2 \vdash S(M_1 + M_2) : \text{Nat}$$

donde:

- $\mathbb{W}(U_1) = \Gamma_1 \vdash M_1 : \sigma_1$
- $\mathbb{W}(U_2) = \Gamma_2 \vdash M_2 : \sigma_2$
- $S = \text{mgu}(\{\sigma_1 \stackrel{?}{=} \text{Nat}, \sigma_2 \stackrel{?}{=} \text{Nat}\} \cup \{\tau \stackrel{?}{=} \rho \mid x : \tau \in \Gamma_1 \wedge x : \rho \in \Gamma_2\})$

Inferir el tipo de la siguiente expresión, o demostrar que no es tipable:

$$\text{foldAT Tern}(\underline{1}, \text{TNil}, x, x) \triangleright \text{TNil} \rightsquigarrow rd; \text{Tern}(x, ri, rm, rd) \rightsquigarrow (ri + rm) + rd$$

- b) Demostrar el siguiente teorema usando deducción natural, sin utilizar principios clásicos:

$$(\exists X. P(X)) \Rightarrow (\forall Y. (P(Y) \Rightarrow Q(Y))) \Rightarrow \exists Z. Q(Z)$$

PLP - Segundo Recuperatorio - 2^{do} cuatrimestre de 2024

#Orden	Nro. Libreta	Apellido y Nombre	E11	E12	E13	Nota Final

Este examen se aprueba obteniendo al menos dos ejercicios bien (B) y uno regular (R), y se promociona con al menos dos ejercicios muy bien (MB) y uno bien (B). Es posible obtener una aprobación condicional con un ejercicio muy bien (MB), uno bien (B) y uno insuficiente (I), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: -, I, R, B, MB, E. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Resolución

a) Dada la siguiente base de conocimientos en Prolog:

```
member(X, [X|_]).
member(X, [_|XS]) :- member(X, XS).

natural(cero).
natural(succ(X)) :- natural(X).
```

```
listaDeNats([ ]).
listaDeNats([X|XS]) :- natural(X), listaDeNats(XS).
```

Explicar brevemente con palabras qué sucede al realizar la siguiente consulta:

```
listaDeNats(XS), member(succ(cero), XS).
```

- Expresar la base de conocimientos y la consulta anterior como fórmulas lógicas.
- Pasar las fórmulas a forma clausal y encontrar una solución a la consulta utilizando resolución.
- La resolución utilizada en el punto anterior ¿fue SLD? Justificar.

Ejercicio 2 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar cut (!) ni predicados de alto orden como `setof`, con la única excepción de `not`.

a) Implementar el predicado `caminoDesde(+P, -C)` que es verdadero cuando C es un camino en un tablero infinito a partir de una posición inicial P. Representaremos a los caminos como una lista de posiciones (X,Y), donde X e Y son números enteros. En cada paso hay cuatro direcciones posibles: arriba, abajo, izquierda y derecha. Por ejemplo

```
?- caminoDesde((0,0),C).
C = [(0, 0)] ;
C = [(0, 0), (0, 1)] ;
C = [(0, 0), (0, -1)] ;
C = [(0, 0), (1, 0)] ;
C = [(0, 0), (-1, 0)] ;
C = [(0, 0), (0, 1), (0, 2)] ;
C = [(0, 0), (0, 1), (0, 0)] ;
C = [(0, 0), (0, 1), (1, 1)] ;
C = [(0, 0), (0, 1), (-1, 1)] ;
C = [(0, 0), (0, -1), (0, 0)] ;
C = [(0, 0), (0, -1), (0, -2)] ;
...
```

b) Trabajaremos con el conocido problema de optimización de la mochila. Consideremos definidos los hechos `objeto(?Id, ?P, ?V)` que instancian diversos objetos con identificador Id, peso P y valor V. Implementar el predicado `mochila(+C, -L)` que es verdadero cuando L es una lista de identificadores de objetos que al guardarlos en la mochila no supera su capacidad C. Por ejemplo, supongamos definidos los siguientes objetos:

```
objeto(1,50,10).
objeto(2,75,15).
objeto(3,60,5).
objeto(4,10,1).
```

Luego, podemos realizar la siguiente consulta:

```
?- mochila(70,L).
L = [] ;
L = [1] ;
L = [1,4] ;
L = [3] ;
L = [3,4] ;
L = [4].
```

Notar que consideramos a las listas como conjuntos, por lo tanto no se deben devolver soluciones repetidas, es decir, listas que contengan los mismos elementos en distinto orden. El orden de los elementos dentro de la lista es arbitrario.

- c) Definir el predicado `mejorMochila(+C,-L)` que es verdadero cuando `L` es una lista de identificadores de objetos que se pueden guardar en la mochila tal que no superen la capacidad `C` y maximicen el valor. Puede haber más de una solución. Por ejemplo:

```
?- mejorMochila(70,L).
L = [1,4] ;
false.
```

Ejercicio 3 - Inferencia y Objetos

- a) Consideremos extendido el Cálculo Lambda (con listas) para representar *RoseTrees*. Para eso se extiende el conjunto de tipos y el de términos de la siguiente manera:

$$\sigma ::= \dots \mid \text{RT}_\sigma \quad M ::= \dots \mid \text{Rose}(M, M) \mid \text{Map } x : \sigma \mapsto M \text{ in } M$$

Los *RoseTrees* son los usuales, mientras que $\text{Map } x : \sigma \mapsto M \text{ in } N$ representa un *RoseTree* con la misma estructura que N , pero que en cada nodo contiene el resultado de reemplazar las apariciones libres de x en M por el valor del nodo correspondiente en N , como se ve en el siguiente ejemplo:

$$\text{Map } t : \text{Nat} \mapsto \text{succ}(t) \text{ in } \text{Rose}(\underline{0}, \text{Rose}(\underline{1}, [])) :: [\text{RT}_{\text{Nat}}] \rightsquigarrow \text{Rose}(\underline{1}, \text{Rose}(\underline{2}, [])) :: [\text{RT}_{\text{Nat}}]$$

El algoritmo de inferencia se extiende de la siguiente manera:

$$\mathcal{W}(\text{Rose}(U, V)) \stackrel{\text{def}}{=} \mathcal{S}\Gamma_1 \cup \mathcal{S}\Gamma_2 \vdash \mathcal{S}(\text{Rose}(M, N)) : \mathcal{S} \text{RT}_\sigma$$

donde:

- $\mathcal{W}(U) = \Gamma_1 \vdash M : \sigma$
- $\mathcal{W}(V) = \Gamma_2 \vdash N : \tau$
- $\mathcal{S} = \text{mgu}(\{\tau \stackrel{?}{=} [\text{RT}_\sigma]\} \cup \{\rho_1 \stackrel{?}{=} \rho_2 \mid x : \rho_1 \in \Gamma_1 \wedge x : \rho_2 \in \Gamma_2\})$

$$\mathcal{W}(\text{Map } x \mapsto U \text{ in } V) \stackrel{\text{def}}{=} \mathcal{S}\Gamma_{1'} \cup \mathcal{S}\Gamma_2 \vdash \mathcal{S}(\text{Map } x : \tau_x \mapsto M \text{ in } N) : \mathcal{S} \text{RT}_{\sigma_1}$$

donde:

- $\mathcal{W}(U) = \Gamma_1 \vdash M : \sigma_1$
- $\mathcal{W}(V) = \Gamma_2 \vdash N : \sigma_2$
- $\Gamma_{1'} = \Gamma_1 \ominus \{x\}$
- $\mathcal{S} = \text{mgu}(\{\sigma_2 \stackrel{?}{=} \text{RT}_{\tau_x}\} \cup \{\rho_1 \stackrel{?}{=} \rho_2 \mid y : \rho_1 \in \Gamma_{1'} \wedge y : \rho_2 \in \Gamma_2\})$

Inferir el tipo de la siguiente expresión, o demostrar que no es tipable:

$$\text{Map } z \mapsto (\lambda x. \text{iszero}(x)) z \text{ in } \text{Rose}(\underline{1}, z)$$

- b) Considere las siguientes clases:

Object subclass: #X

```
action1
  ^[self compute] value.
```

```
compute
  ^10.
```

```
baseValue
  ^self value + 5.
```

```
value
  ^1.
```

X subclass: #Y

```
action2
  ^super baseValue.
```

```
baseValue
  ^20.
```

```
value
  ^3.
```

Para cada una de las siguientes expresiones, hacer una tabla donde se indique, en orden, cada mensaje que se envía, qué objeto lo recibe, en qué clase está el método respectivo, y cuál es el resultado final de cada colaboración:

- i) `Y new action1`
- ii) `Y new action2`

PLP - Segundo Parcial - 1^{er} cuatrimestre de 2025 - Parte 1

#Orden	Libreta	Apellido y Nombre	Nota
			A

Esta parte del examen se aprueba respondiendo correctamente al menos 5 de las 8 preguntas. Es necesario aprobar esta parte para acceder a la corrección de la segunda parte. Responder las preguntas en los espacios indicados.

Pregunta 1: Sea la extensión del Cálculo Lambda con listas y sumas:

$$M ::= \dots \mid M + M \mid []_r \mid M :: M \mid \text{foldr } M \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow M$$

Rectificar el siguiente término: $(\lambda x. \text{foldr } h \text{ base } \rightsquigarrow \text{succ}(x); \text{rec}(h, r) \rightsquigarrow \text{succ}(h) + r) r$

Respuesta: $(\lambda x. \text{foldr } h \text{ base } \rightarrow \text{succ}(x); \text{rec}(h, r) \rightarrow \text{succ}(h) + r) r$ ✓

Pregunta 2: Indicar cuál sería un paso válido del algoritmo de MGU en el siguiente conjunto de ecuaciones de unificación: $\{c \stackrel{?}{=} X_1, [X_2] \stackrel{?}{=} [X_3]\}$

- a. $\xrightarrow{\text{elim}}_{\{[X_2] := [X_3]\}} \{c \stackrel{?}{=} X_1\}$ b. $\xrightarrow{\text{elim}}_{\{c := X_1\}} \{[X_2] \stackrel{?}{=} [X_3]\}$ c. $\xrightarrow{\text{swap}} \{X_1 \stackrel{?}{=} c, [X_2] \stackrel{?}{=} [X_3]\}$

Respuesta: c. (Ancho que c es una CTE) ✓

Pregunta 3: Indicar la sustitución resultante de aplicar el algoritmo de MGU en el siguiente conjunto de ecuaciones de unificación: $\{X_1 \stackrel{?}{=} X_2 \rightarrow X_3, [X_2] \stackrel{?}{=} [X_4]\}$

Respuesta: _____

Pregunta 4: Se quiere demostrar en deducción natural la siguiente fórmula:

$$\exists X. \forall Y. (P(X) \wedge Q(Y)) \vdash \exists Z. (P(Z) \wedge Q(Z))$$

Renombramos $\Gamma = \exists X. \forall Y. (P(X) \wedge Q(Y))$. Indicar con cuál de las siguientes reglas se puede avanzar en la demostración y cuáles serían las premisas:

- a. $\exists_i : \frac{\Gamma \vdash P(Z) \wedge Q(Z)}{\dots}$
 b. $\exists_i : \frac{\Gamma \vdash P(X) \wedge Q(X)}{\dots}$
 c. $\exists_e : \frac{\Gamma \vdash \exists Z. \forall Y. (P(Z) \wedge Q(Y)) \quad \Gamma, \forall Y. (P(Y) \wedge Q(Y)) \vdash \exists Z. (P(Z) \wedge Q(Z))}{\dots}$
 d. Ninguna de las anteriores (no es válido)

Respuesta: c. ✓

Pregunta 5: Sea la siguiente base de conocimiento en Prolog:

p(a).
 q(a).
 q(b).

¿Cuál es el resultado de la siguiente consulta? $\text{not}(p(X)), q(X).$

☐ X = a; X = b. ☐ X = b. ☐ true. ☒ false.

Respuesta: _____

Pregunta 6: Sea la representación de árboles con los átomos `nil` y `bin(I,N,D)`. Se quiere definir un predicado que instancie **todos** los posibles árboles con los nodos sin instanciar. Indicar cuál de las siguientes opciones es correcta:

- a. `arbol(nil).`
`arbol(bin(I,_,D)) :- arbol(I), arbol(D).`
- b. `arbol(nil).`
`arbol(A) :- A = nil, arbol(bin(I,_,D)).`
- c. Las dos son correctas
- d. Ninguna es correcta

Respuesta: b

Pregunta 7: ¿Cuál de las siguientes definiciones de predicados instancia en el segundo argumento de `p` la longitud de la lista al realizar la consulta `p([1,2,3],X)`?

- a. `p(L,X) :- X is length(L).`
- b. `p(L,X) :- length(L,X).`
- c. `p(L,length(L)).`
- d. Ninguna de las anteriores

Respuesta: b

Pregunta 8: Dadas las siguientes cláusulas:

1. $\{P(X), Q(X), S(c)\}$
2. $\{\neg P(X), \neg Q(X), R(X)\}$
3. $\{Q(a), Q(X)\}$

Indicar cuál es una posible resolvente.

- a. $\{S(c), R(X)\}$ (de 1 y 2)
- b. $\{\neg P(a), R(a)\}$ (de 2 y 3)
- c. $\{P(a), S(c)\}$ (de 1 y 3)
- d. $\{\neg P(X), R(X), Q(X)\}$ (de 2 y 3)

Respuesta: d

PLP - Segundo Parcial - 1^{er} cuatrimestre de 2025

#Orden	Nro. Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final

Este examen se aprueba obteniendo al menos dos ejercicios bien (B) y uno regular (R), y se promociona con al menos dos ejercicios muy bien (MB) y uno bien (B). Es posible obtener una aprobación condicional con un ejercicio muy bien (MB), uno bien (B) y uno insuficiente (I), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: -, I, R, B, MB, E. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar cut (!) ni predicados de alto orden como `setof`, con la única excepción de `not`. No está permitido usar macros que colapsen cláusulas.

- a) Definir el predicado `esRotación(+L,+R)` que es verdadero cuando R es una rotación de L. Una lista R es una rotación de una lista L si contiene los mismos elementos en el mismo orden relativo, pero comenzando desde una posición diferente dentro de L, recorriendo la lista circularmente. Por ejemplo:

```
?- esRotacion([1,2,3,4], [3,4,1,2]).
true.
?- esRotacion([a,b,c], [c,a,b]).
true.
?- esRotacion([a,b,c], [c,b,a]).
false.
```

- b) La *secuencia de Collatz* se construye con la siguiente regla sobre un número natural: si n es par, el siguiente número es $n/2$; si n es impar, el siguiente número es $3n + 1$. La secuencia termina cuando se llega al número 1 (aunque la conjetura dice que siempre se llega a 1, esto no ha sido probado para todos los naturales).

Definir el predicado `collatz(+N,-S)` que instancia en S todos los números pertenecientes a la secuencia de Collatz a partir del número N. Por ejemplo:

```
?- collatz(6,S).
S = 6 ;
S = 3 ;
S = 10 ;
S = 5 ;
S = 16 ;
S = 8 ;
S = 4 ;
S = 2 ;
S = 1 ;
false.
```

- c) ¿El predicado anterior es reversible en S? (pueden asumir que se utilizarán números bajos por lo que la secuencia es siempre finita). Justificar brevemente.
- d) Definir el predicado `collatzMayor(+N,-M)` que es verdadero cuando M es el número más grande de la secuencia de Collatz a partir del número N. Por ejemplo:

```
?- collatzMayor(6,M).
M = 16 ;
false.
```

Ejercicio 2 - Resolución

- a. Dada la siguiente base de conocimientos en Prolog:

```
member(X,[X|_]).
member(X,[Y|XS]) :- member(X,XS).

esSublista(_,[]).
esSublista(L,[X|XS]) :- member(X,L), esSublista(L,XS).
```

Explicar **brevemente** con palabras qué sucede al realizar la siguiente consulta:

`esSublista([a,b],R), member(b,R).`

- Expresar la base de conocimientos y la consulta anterior como fórmulas lógicas.
- Pasar las fórmulas a forma clausal y encontrar una solución a la consulta utilizando resolución.
- La resolución utilizada en el punto anterior ¿fue SLD? Justificar.
- ¿Hay alguna diferencia entre su resolución y en cómo resuelve Prolog la consulta? Si es así, explicar cuál.

Ejercicio 3 - Inferencia, Deducción Natural y POO

- Consideremos el Cálculo Lambda tipado extendido con árboles binarios con información en las hojas de la siguiente manera:

$\tau ::= \dots \mid \text{AIH}(\tau)$

$M ::= \dots \mid \text{Hoja}(M) \mid \text{Bin}(M, M) \mid \text{foldAIH } M \triangleright \text{Hoja}(x) \rightsquigarrow M; \text{Bin}(i, d) \rightsquigarrow M$

- $\text{AIH}(\tau)$ es el tipo de los árboles con información en las hojas de tipo τ .
- $\text{Hoja}(M)$ es un árbol compuesto por una única hoja con información M .
- $\text{Bin}(M_1, M_2)$ es un árbol compuesto por dos subárboles M_1 y M_2 .
- $\text{foldAIH } M_1 \triangleright \text{Hoja}(x) \rightsquigarrow M_2; \text{Bin}(i, d) \rightsquigarrow M_3$ representa el esquema de recursión estructural sobre árboles con los casos de hoja (en donde la variable x se liga con el valor de la hoja y puede aparecer libre en M_2) y bin (que ligará las variables i y d al resultado de la recursión sobre sus subárboles y pueden aparecer libres en M_3).

Se extiende también el algoritmo de inferencia de la siguiente manera:

$\mathcal{I}(\Gamma \mid \text{Hoja}(M)) = (\text{AIH}(\tau) \mid E) \quad \text{donde } \mathcal{I}(\Gamma \mid M) = (\tau \mid E)$

$\mathcal{I}(\Gamma \mid \text{Bin}(M_1, M_2)) = (\tau_1 \mid \{\tau_2 \stackrel{?}{=} \text{AIH}(X_h), \tau_1 \stackrel{?}{=} \tau_2\} \cup E_1 \cup E_2)$
donde $\mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1)$
 $\mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2)$
 X_h variable fresca

$\mathcal{I}(\Gamma \mid \text{foldAIH } M_1 \triangleright \text{Hoja}(x) \rightsquigarrow M_2; \text{Bin}(i, d) \rightsquigarrow M_3) =$
 $(\tau_2 \mid \{\tau_1 \stackrel{?}{=} \text{AIH}(X_x), \tau_2 \stackrel{?}{=} \tau_3, X_i \stackrel{?}{=} \tau_2, X_d \stackrel{?}{=} \tau_2\} \cup E_1 \cup E_2 \cup E_3)$
donde $\mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1)$
 $\mathcal{I}(\Gamma, x : X_x \mid M_2) = (\tau_2 \mid E_2)$
 $\mathcal{I}(\Gamma, i : X_i, d : X_d \mid M_3) = (\tau_3 \mid E_3)$
 X_x, X_i y X_d variables frescas

Usar el algoritmo $\mathcal{I}$ para inferir el tipo de la siguiente expresión, o demostrar que no es tipable:

`foldAIH Bin(Hoja(1), Hoja(x))  $\triangleright$  Hoja(x)  $\rightsquigarrow$  isZero(x); Bin(x, y)  $\rightsquigarrow$  x`

- Demostrar el siguiente teorema usando deducción natural, sin utilizar principios clásicos:

$$((\exists X. \neg P(X)) \wedge (\forall X. \neg R(X))) \Rightarrow \exists X. \neg (P(X) \wedge Q(X))$$

- Implementar el método `#twice` cuyo objeto receptor es un bloque de un parámetro y devuelve un nuevo bloque que aplica el original dos veces. Indicar también dónde debería ir definido el método. Por ejemplo, la siguiente ejecución evalúa a 12:

```
| double doubleTwice |
double := [:x | x * 2].
doubleTwice := double twice.
doubleTwice value: 3.
```

PLP - Segundo Recuperatorio - 1^{er} cuatrimestre de 2025

#Orden	Turno	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final

Este examen se aprueba obteniendo al menos dos ejercicios bien (B) y uno regular (R), y se promociona con al menos dos ejercicios muy bien (MB) y uno bien (B). Es posible obtener una aprobación condicional con un ejercicio muy bien (MB), uno bien (B) y uno insuficiente (I), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: -, I, R, B, MB, E. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar macros que colapsen cláusulas, cut (!) ni predicados de alto orden, con la única excepción de not. No está permitido el uso de estructuras auxiliares.

- a) Definir el predicado `unico(+L,-U)` que es verdadero cuando U aparece una sola vez en la lista L. Resolver sin contar apariciones. Por ejemplo:

```
?- unico([a,b,c,a,a,b,d],U).
U = c ;
U = d ;
false.
```

- b) Definir el predicado `sinRepetidos(+L)` que represente la siguiente propiedad: *una lista L hace verdadero a `sinRepetidos` si todos sus elementos son únicos.*

- c) En este inciso vamos a representar a un subconjunto de las fórmulas de lógica proposicional con negación e implicación. Los átomos utilizados serán `neg(P)` e `imp(P,Q)`, con P y Q fórmulas. Definir el predicado `formula(+VS,-F)` que instancia en F todas las fórmulas que usan las variables proposicionales que aparecen en la lista VS. Por ejemplo:

<code>?- formula([p,q],F).</code>	<code>F = neg(neg(p));</code>	<code>F = neg(imp(q,q));</code>
<code>F = p;</code>	<code>F = neg(neg(q));</code>	<code>F = imp(p,neg(p));</code>
<code>F = q;</code>	<code>F = imp(p,p);</code>	<code>...</code>
<code>F = neg(p);</code>	<code>F = imp(p,q);</code>	<code>F = imp(neg(q),imp(neg(p),p));</code>
<code>F = neg(q);</code>	<code>...</code>	<code>...</code>

Sugerencia: prestar atención al orden en que se devuelven los resultados. No es necesario que se mantenga el orden del ejemplo, pero sí que se instancien todas las fórmulas.

- d) ¿El predicado anterior es reversible en F? Justificar brevemente.

Ejercicio 2 - Resolución

Consideremos el siguiente lenguaje del dominio de funciones numéricas:

- $X \leq Y$: predicado binario que representa que X es menor o igual que Y
- $cota(F, X)$: predicado binario que representa que X es una cota superior de la función F
- $sup(F)$: símbolo de función unario que representa el supremo de la función F
- $ev(F, N)$: símbolo de función binario que representa el resultado de evaluar la función F en N

Sean los siguientes axiomas:

- $\forall F. \forall X. (cota(F, X) \Leftrightarrow \forall N. ev(F, N) \leq X)$
(definición de cota superior)
- $\forall F. (cota(F, sup(F)) \wedge \forall Y. (cota(F, Y) \Rightarrow sup(F) \leq Y))$
(el supremo de una función es una cota superior, y es la más chica de las cotas superiores)
- $\forall X. \forall Y. \forall Z. ((X \leq Y \wedge Y \leq Z) \Rightarrow X \leq Z)$
(transitividad de la relación menor o igual)

Se pide:

- Pasar a forma clausal cada fórmula
- Demostrar usando resolución que si una función es más chica que otra en todo punto, el supremo también es más chico:

$$\forall F. \forall G. ((\forall N. ev(F, N) \leq ev(G, N)) \Rightarrow sup(F) \leq sup(G))$$
- La resolución utilizada en el punto anterior ¿fue SLD? Justificar.

Ejercicio 3 - Inferencia, Deducción Natural y POO

- Consideremos el Cálculo Lambda tipado extendido con intervalos de números naturales:

$\sigma ::= \dots \mid \text{Intervalo}$

$M ::= \dots \mid [M, M] \mid \text{case } M \text{ of } [] \rightsquigarrow M; x :: y \rightsquigarrow M$

donde $[M, N]$ es el intervalo que va desde el número M hasta N (inclusive) y $\text{case } M \text{ of } [] \rightsquigarrow N; x :: y \rightsquigarrow O$ es un observador que recorre el intervalo M como si fuese una lista: con un caso para el intervalo vacío (N) y otro para un intervalo con al menos un elemento (O). En el segundo caso, las apariciones libres de la variable x en O se ligarán al primer elemento del intervalo y las apariciones libres de y en O se ligarán al intervalo sin su primer elemento.

Se extiende también el algoritmo de inferencia de la siguiente manera:

$\mathcal{I}(\Gamma \mid [M_1, M_2]) = (\text{Intervalo} \mid \{\tau_1 \stackrel{?}{=} \text{Nat}, \tau_1 \stackrel{?}{=} \tau_2\} \cup E_1 \cup E_2)$
donde $\mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1)$
 $\mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2)$

$\mathcal{I}(\Gamma \mid \text{case } M_1 \text{ of } [] \rightsquigarrow M_2; x :: y \rightsquigarrow M_3) =$
 $(\tau_2 \mid \{\tau_2 \stackrel{?}{=} \tau_3, \tau_1 \stackrel{?}{=} \text{Intervalo}, X_x \stackrel{?}{=} \text{Nat}, X_y \stackrel{?}{=} \text{Intervalo}\} \cup E_1 \cup E_2 \cup E_3)$
donde $\mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1)$
 $\mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2)$
 $\mathcal{I}(\Gamma, x : X_x, y : X_y \mid M_3) = (\tau_3 \mid E_3)$
 X_x y X_y variables frescas

Usar el algoritmo $\mathcal{I}$ para inferir el tipo de la siguiente expresión, o demostrar que no es tipable:

$\text{case } (\lambda x. [\underline{1}, x]) y \text{ of } [] \rightsquigarrow (\lambda z. [y, y]); x :: z \rightsquigarrow z$

- Demostrar el siguiente teorema usando deducción natural, sin utilizar principios clásicos:

$$(\forall X. \exists Y. (P(X, Y) \wedge Q(Y, X))) \Rightarrow \exists X. \exists Y. Q(Y, X)$$

- En una empresa, al final del día se realizan varias tareas rutinarias sobre distintas estructuras de datos. Estas tareas se encapsulan en bloques de código y deben ejecutarse secuencialmente sobre un mismo objeto. Se desea implementar un mecanismo genérico que permita realizar esta secuencia de operaciones sobre cualquier objeto.

Implementar en la clase `Object` el método `runBatch`, que reciba como parámetro una colección de bloques. Cada bloque debe tomar el objeto receptor como argumento, y se debe ejecutar en orden. El método debe devolver el resultado de la ejecución del último bloque.

Por ejemplo:

```
| lista resultado |
lista := OrderedCollection new.
resultado := lista runBatch: #(
  [ :obj | obj add: 1 ]
  [ :obj | obj add: 2 ]
  [ :obj | 'último paso' ])
```

Al final de la ejecución, `resultado` es 'último paso', mientras que `lista` contiene 1 y 2.

PLP - Segundo Parcial - 2^{do} cuatrimestre de 2025

#Orden	Turno	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
				E	E	E	P

Este examen se aprueba obteniendo al menos dos ejercicios bien (B) y uno regular (R), y se promociona con al menos dos ejercicios muy bien (MB) y uno bien (B). Es posible obtener una aprobación condicional con un ejercicio muy bien (MB), uno bien (B) y uno insuficiente (I), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: -, I, R, B, MB, E. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Entregar cada ejercicio en hojas separadas. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar macros que colapsen cláusulas, cut (!) ni predicados de alto orden, con la única excepción de not.

En este ejercicio trabajaremos con melodías. Se cuenta con el predicado `nota(?N)` que es verdadero cuando N es una nota (do, re, mi, etc). Las melodías pueden ser notas o construirse con el término `sec(M1, M2)`, que representa una secuencia de dos melodías. Así, un ejemplo de melodía podría ser `sec(sec(do, fa), la)`, que representa la melodía do, fa, la.

- a) Definir el predicado `listaAMelodia(+L, -M)` que es verdadero cuando M es una melodía que contiene en orden las notas de la lista L. Por ejemplo:

```
?- listaAMelodia([do, re, mi, fa], M).
M = sec(do, sec(re, sec(mi, fa)));
M = sec(do, sec(sec(re, mi), fa));
M = sec(sec(do, re), sec(mi, fa));
M = sec(sec(do, sec(re, mi)), fa);
M = sec(sec(sec(do, re), mi), fa);
false.
```

- b) Definir el predicado `submelodia(+M, -S)` que es verdadero cuando S es una subsecuencia de M. Una melodía S es subsecuencia de una melodía M cuando S es un subtérmino dentro de la estructura de M, obtenido siguiendo algún camino de `sec` sin reordenar ni omitir partes. Es decir, la melodía `sec(do, sec(re, mi))` tiene como subsecuencias a `sec(re, mi)` y `sec(do, sec(re, mi))`, pero no son subsecuencias `sec(do, re)`, `sec(do, mi)`, `do`, `re` ni `mi`. Por ejemplo:

```
?- submelodia(sec(sec(do, sec(re, mi)), fa), S).
S = sec(sec(do, sec(re, mi)), fa);
S = sec(do, sec(re, mi));
S = sec(re, mi);
false.
```

¿El predicado es reversible en S? Justificar brevemente.

- c) Definir el predicado `sinSubmelodiasEnComun(+M1, +M2)` que es verdadero cuando M1 y M2 no tienen submelodías en común. Por ejemplo:

```
?- sinSubmelodiasEnComun(sec(do, sec(re, mi)), sec(fa, sec(re, mi))).
false.
```

```
?- sinSubmelodiasEnComun(sec(do, sec(re, mi)), sec(fa, sec(re, sol))).
true.
```

- d) Definir el predicado `melodia(-M)` que instancia en M todas las melodías. No se permite definir `melodia` en función de `listaAMelodia`, debe definirse generando las estructuras en cada paso. Por ejemplo, suponiendo que las únicas notas fueran do y re:

```
?- melodia(M).
M = do;
M = re;
M = sec(do, do);
```

```
M = sec(do, re);
M = sec(re, do);
M = sec(re, re);
M = sec(do, sec(do, do));
```

```
M = sec(do, sec(do, re));
...
M = sec(sec(do, re), do);
...
```


Ejercicio 2 - Resolución

a. Representar en forma clausal las siguientes fórmulas de lógica de primer orden:

- $\forall X. (\text{vacío}(X) \Leftrightarrow \forall Z. \neg(Z \in X))$
Un conjunto es vacío si y solo si ningún elemento pertenece a él.
- $\forall X. \forall Y. \forall Z. (Z \in X \vee Z \in Y \Rightarrow Z \in X \cup Y)$
Si un elemento pertenece a un conjunto o a otro, entonces pertenece a la unión de ambos.

b. Escribir brevemente un plan para demostrar la siguiente propiedad:

$\forall X. \forall Y. (\text{vacío}(X \cup Y) \Rightarrow \text{vacío}(X) \wedge \text{vacío}(Y))$
Si la unión de dos conjuntos es vacía, entonces ambos son vacíos.

- Utilizar resolución para demostrar la fórmula del inciso anterior. Indicar la sustitución utilizada en cada paso.
- La resolución utilizada en el punto anterior, ¿fue SLD? Justificar.

Ejercicio 3 - Inferencia y Deducción Natural

a. Consideremos el Cálculo Lambda tipado extendido con colas (queues) de la siguiente manera:

$\tau ::= \dots \mid \text{Cola}_\tau$

$M ::= \dots \mid \langle \rangle_\tau \mid M \bullet M \mid \text{map } x : \tau \mapsto M \text{ in } M$

- Cola_τ es el tipo de las colas de tipo τ .
- $\langle \rangle_\tau$ es una cola vacía de tipo τ .
- $M_1 \bullet M_2$ es una cola M_1 a la cual se le agrega al final un elemento M_2 .
- $\text{map } e : \sigma \mapsto M_1 \text{ in } M_2$ representa una cola con la misma estructura que M_2 , pero que en cada elemento contiene el resultado de reemplazar las apariciones libres de e en M_1 , por el valor del elemento correspondiente en M_2 , como se ve en el siguiente ejemplo:

$$\text{map } e : \text{Nat} \mapsto \text{succ}(e) \text{ in } \langle \rangle_{\text{Nat}} \bullet 3 \bullet 0 \bullet 1 \rightsquigarrow \langle \rangle_{\text{Nat}} \bullet 4 \bullet 1 \bullet 2$$

Se dispone de las siguientes reglas de tipado:

$$\frac{}{\Gamma \vdash \langle \rangle_\tau : \text{Cola}_\tau}$$

$$\frac{\Gamma \vdash M_1 : \text{Cola}_\tau \quad \Gamma \vdash M_2 : \tau}{\Gamma \vdash M_1 \bullet M_2 : \text{Cola}_\tau}$$

$$\frac{\Gamma, x : \tau \vdash M_1 : \sigma \quad \Gamma \vdash M_2 : \text{Cola}_\tau}{\Gamma \vdash \text{map } x : \tau \mapsto M_1 \text{ in } M_2 : \text{Cola}_\sigma}$$

Se extiende también el algoritmo de inferencia de la siguiente manera:

$$\mathcal{I}(\Gamma \mid \langle \rangle_\tau) = (\text{Cola}_\tau \mid \emptyset)$$

$$\mathcal{I}(\Gamma \mid M_1 \bullet M_2) = (\text{Cola}_{\tau_2} \mid \{\tau_1 \stackrel{?}{=} \text{Cola}_{\tau_2}\} \cup E_1 \cup E_2)$$

donde $\mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1)$
 $\mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2)$

$$\mathcal{I}(\Gamma \mid \text{map } x : \tau_x \mapsto M_1 \text{ in } M_2) = (\text{Cola}_{\tau_1} \mid \{\tau_2 \stackrel{?}{=} \text{Cola}_{\tau_x}\} \cup E_1 \cup E_2)$$

donde $\mathcal{I}(\Gamma, x : \tau_x \mid M_1) = (\tau_1 \mid E_1)$
 $\mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2)$

Se pide:

i) Rectificar el siguiente término:

$$(\lambda y. \text{map } x \mapsto \text{if } x \text{ then } y \text{ else } 0 \text{ in } \langle \rangle \bullet \text{true} \bullet x) (y \ x)$$

ii) Usar el algoritmo $\mathcal{I}$ para inferir el tipo de las siguientes expresiones, o demostrar que no son tipables:

$$(\lambda x. \lambda y. z)(\langle \rangle \bullet \text{true})$$

$$(\text{map } e \mapsto \text{isZero}(e) \text{ in } \langle \rangle \bullet 1)$$

No es necesario escribir el paso a paso del MGU, pero deben estar presentes las ecuaciones de unificación y la sustitución final.

b. Demostrar el siguiente teorema en deducción natural. Es necesario usar principios clásicos. Recordar que pueden utilizar los teoremas demostrados en la guía (de la forma que están escritos allí).

$$(\neg \forall Y. P(Y)) \Rightarrow \exists X. (P(X) \Rightarrow Q(X))$$

PLP - Segundo Recuperatorio - 2^{do} cuatrimestre de 2025

#Orden	Turno	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
#322	N		ROSSIGNOL, Santiago	MB	E	E	P

Este examen se aprueba obteniendo al menos dos ejercicios bien (B) y uno regular (R), y se promociona con al menos dos ejercicios muy bien (MB) y uno bien (B). Es posible obtener una aprobación condicional con un ejercicio muy bien (MB), uno bien (B) y uno insuficiente (I), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: -, I, R, B, MB, E. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Entregar cada ejercicio en hojas separadas. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar `cut (!)` ni predicados de alto orden como `setof`, con la única excepción de `not`. No está permitido usar macros que colapsen cláusulas.

En este ejercicio trabajaremos con matrices, representándolas como listas de listas, en donde cada lista representa en orden las filas de la matriz. Cada matriz debe tener al menos una celda, y no puede haber filas vacías.

- a) Definir el predicado `secuenciaRepetida(+M,-S)` que es verdadero cuando S es una secuencia de elementos consecutivos que aparece en todas las filas de M. Notar que la lista vacía no se considera subsecuencia. Por ejemplo:

```
?- sequenciaRepetida([[1,2,3],[1,2,4],[3,1,2]],S).
S = [1];
S = [1,2];
S = [2];
false.
```

- b) Definir el predicado **secuenciaMáxima**(+M,-S) que instancia en S la secuencia de elementos consecutivos de mayor longitud que se repite en todas las filas de la matriz M. Por ejemplo:

```
?- secuenciaMáxima([[a,b,c,d],[b,c,d,e],[a,b,c,e]],S).
S = [b,c];
false.

?- secuenciaMáxima([[1,2,3],[1,3,5]],S).
S = [1];
S = [3];
false.
```

- c) Definir el predicado `todasLasMatrices(-M)` que instancia en `M` **todas** las matrices válidas con sus elementos sin instanciar. Por ejemplo:

```
?- todasLasMatrices(M).      M = [[_,_,_]];           M = [[_,_,_],[_,_,_]];
M = [[_]];                  M = [[_,_],[_,_]];       M = [[_,_],[_,_],[_,_]];
M = [[_,_]];                M = [[_],[_],[_]];       M = [[_],[_],[_],[_]];
M = [[_],[_]];              M = [[_,_,_,_]];       ...
```

- d) ¿El predicado anterior es reversible? Justificar.

Ejercicio 2 - Resolución

- a. Dada la siguiente base de conocimientos en Prolog:

```

nota(do).
submelodia(sec(M,N),sec(M,N)).
submelodia(sec(M,_),S) :- submelodia(M,S).
submelodia(sec(_,N),S) :- submelodia(N,S).
melodia(M) :- nota(M).
melodia(sec(M,N)) :- melodia(M), melodia(N).

```

Explicar **brevemente** con palabras qué sucede al realizar la siguiente consulta:

```
melodía(M), submelodía(M,sec(sec(do,do),do)).
```

- b. Expresar la base de conocimientos y la consulta anterior como fórmulas lógicas.

- c. Pasar las fórmulas a forma clausal y encontrar una solución a la consulta utilizando resolución.
Sugerencia: pensar cuál debería ser el resultado para armar el plan.
- d. La resolución utilizada en el punto anterior ¿fue SLD? Justificar.
- e. ¿Hay alguna diferencia entre su resolución y en cómo resuelve Prolog la consulta? Si es así, detallar cuál.

Ejercicio 3 - Inferencia y Deducción Natural

a) Consideremos el Cálculo Lambda tipado extendido con regiones geométricas:

$\tau ::= \dots \mid \text{Región}$

$M ::= \dots \mid \text{Rectángulo}(M, M) \mid \text{Unión}(M, M) \mid \text{foldRG } M \triangleright \text{Rectángulo}(x, y) \rightsquigarrow M; \text{Unión}(ri, rd) \rightsquigarrow M$

donde

- Región es el tipo de las regiones del plano formadas por rectángulos y uniones de rectángulos.
- $\text{Rectángulo}(M_1, M_2)$ es un rectángulo de ancho dado por M_1 y alto dado por M_2 .
- $\text{Unión}(M_1, M_2)$ es la región del plano formada por la unión de las regiones M_1 y M_2 .
- $\text{foldRG } M_1 \triangleright \text{Rectángulo}(x, y) \rightsquigarrow M_2; \text{Unión}(ri, rd) \rightsquigarrow M_3$ es el esquema de recursión estructural para regiones, donde las variables x e y se ligarán en M_2 al ancho y alto del rectángulo, y ri y rd se ligarán en M_3 a los resultados de la recursión sobre las regiones unidas.

Se dispone de las siguientes reglas de tipado:

$$\frac{\Gamma \vdash M_1 : \text{Nat} \quad \Gamma \vdash M_2 : \text{Nat}}{\Gamma \vdash \text{Rectángulo}(M_1, M_2) : \text{Región}} \quad \frac{\Gamma \vdash M_1 : \text{Región} \quad \Gamma \vdash M_2 : \text{Región}}{\Gamma \vdash \text{Unión}(M_1, M_2) : \text{Región}}$$

$$\frac{\Gamma \vdash M_1 : \text{Región} \quad \Gamma, x : \text{Nat}, y : \text{Nat} \vdash M_2 : \tau \quad \Gamma, ri : \tau, rd : \tau \vdash M_3 : \tau}{\Gamma \vdash \text{foldRG } M_1 \triangleright \text{Rectángulo}(x, y) \rightsquigarrow M_2; \text{Unión}(ri, rd) \rightsquigarrow M_3 : \tau}$$

Se extiende también el algoritmo de inferencia de la siguiente manera:

$$\mathcal{I}(\Gamma \mid \text{Rectángulo}(M_1, M_2)) = (\text{Región} \mid \{\tau_1 \stackrel{?}{=} \tau_2, \tau_2 \stackrel{?}{=} \text{Nat}\} \cup E_1 \cup E_2)$$

donde $\mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1)$
 $\mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2)$

$$\mathcal{I}(\Gamma \mid \text{Unión}(M_1, M_2)) = (\text{Región} \mid \{\tau_1 \stackrel{?}{=} \tau_2, \tau_2 \stackrel{?}{=} \text{Región}\} \cup E_1 \cup E_2)$$

donde $\mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1)$
 $\mathcal{I}(\Gamma \mid M_2) = (\tau_2 \mid E_2)$

$$\mathcal{I}(\Gamma \mid \text{foldRG } M_1 \triangleright \text{Rectángulo}(x, y) \rightsquigarrow M_2; \text{Unión}(ri, rd) \rightsquigarrow M_3) =$$

$$(\tau_3 \mid \{\tau_2 \stackrel{?}{=} \tau_3, \tau_1 \stackrel{?}{=} \text{Región}, X_x \stackrel{?}{=} \text{Nat}, X_y \stackrel{?}{=} X_x, X_{ri} \stackrel{?}{=} X_{rd}, X_{rd} \stackrel{?}{=} \tau_3\} \cup E_1 \cup E_2 \cup E_3)$$

donde $\mathcal{I}(\Gamma \mid M_1) = (\tau_1 \mid E_1)$
 $\mathcal{I}(\Gamma, x : X_x, y : X_y \mid M_2) = (\tau_2 \mid E_2)$
 $\mathcal{I}(\Gamma, ri : X_{ri}, rd : X_{rd} \mid M_3) = (\tau_3 \mid E_3)$
 X_x, X_y, X_{ri} y X_{rd} variables frescas

Se pide:

i) Rectificar el siguiente término:

$\text{foldRG Rectángulo}(x, y) \triangleright \text{Rectángulo}(x, y) \rightsquigarrow (\lambda z.x)(z \ y); \text{Unión}(ri, rd) \rightsquigarrow \text{if } y \text{ then } ri \text{ else } rd$

ii) Usar el algoritmo $\mathcal{I}$ para inferir el tipo de las siguientes expresiones, o demostrar que no son tipables:

$\text{Unión}((\lambda x.x) \ y, (\lambda x.x \ z))$

$\text{foldRG Rectángulo}(1, z) \triangleright \text{Rectángulo}(x, y) \rightsquigarrow w; \text{Unión}(ri, rd) \rightsquigarrow \text{isZero}(ri)$

No es necesario escribir el paso a paso del MGU, pero deben estar presentes las ecuaciones de unificación y la sustitución final.

b) Demostrar el siguiente teorema usando deducción natural sin utilizar principios clásicos:

$$\exists X.P(X, X) \Rightarrow \exists Y.\neg\forall Z.\neg P(Y, Z)$$

Sugerencia: pensar cuál es la Y que satisface $\neg\forall Z.\neg P(Y, Z)$